

使用自動調度資源、自動修復和映像檔更新功能，透過 MIG 部署 Confidential Space 工作負載

來源：<https://codelabs.developers.google.com/codelabs/confidential-space-mig?hl=zh-tw>

步驟數：8

在本程式碼研究室中，您將瞭解如何在代管執行個體群組上部署 Confidential Space 工作負載容器。您將設定這個 MIG，納入自動修復的健康狀態檢查、使用 CPU 使用率自動調度資源運算資源，並在 MIG 中為每個 VM 設定 OS 映像檔更新和更新後的工作負載映像檔，以進行滾動式更新。本程式碼研究室著重於工作負載的基本實作。

目錄

1. [1. 總覽](#)
2. [2. 如何使用 MIG 建立 Confidential Space](#)
3. [3. 設定雲端資源](#)
4. [4. 設定工作負載](#)
5. [5. 設定執行個體範本和代管執行個體群組](#)
6. [6. 啟用自動修復和自動調度資源的步驟](#)
7. [7. 驗證工作負載及設定映像檔更新](#)
8. [8. 清除](#)

1. 總覽

Confidential Space (CS) 提供安全、經過認證的加密環境，可處理機密資料。如果依賴獨立 VM 執行個體，會造成作業負擔，因為手動自動化調度管理缺乏重要業務服務所需的擴充性。如果沒有自動化調度管理，要對整個機群執行同步的滾動式更新，或部署新的 OS 映像檔，在技術上會很困難，且容易發生停機時間。

在本程式碼研究室中，您將瞭解如何在代管執行個體群組 (MIG) 上部署 Confidential Space 工作負載。您也會瞭解如何使用健康狀態檢查啟用自動修復功能、根據 CPU 使用率自動調度資源，以及為 OS 映像檔和工作負載啟用滾動式更新。

本程式碼研究室中顯示的程序，應有助於您為任務關鍵型長期部署作業，設定自己的安全 Confidential Space，並準備好投入生產。

課程內容

- 如何為 Confidential Space 建立專用執行個體範本。
- 如何使用 Google Compute Engine，以及如何設定 MIG 和執行個體群組
- 如何建立防火牆規則和健康狀態檢查，以啟用自動修復功能。
- 如何使用範本和健康狀態檢查設定可用區 MIG。
- 如何為 MIG 設定自動調度資源。
- 如何使用 MIG 上的指令碼，為工作負載映像檔和 Confidential Space 的新 OS 映像檔版本，設定一鍵式 OS 映像檔更新

軟硬體需求

- 已啟用計費功能的 Google Cloud 專案。
 - 熟悉文字編輯器、Docker 部署作業和 Bash 指令碼
 - 已安裝並驗證 gcloud 指令列工具。
 - 基本瞭解 Compute Engine、Confidential Space、IAM、Confidential VM、容器技術、遠端存放區、服務帳戶、Cloud Run 和 Cloud Scheduler
 - 已建構並推送至 Artifact Registry 的 Confidential Space 工作負載容器映像檔。
-

2. 如何使用 MIG 建立 Confidential Space

使用代管執行個體群組 (MIG) 部署 Confidential Space 工作負載，可讓安全應用程式更穩固、更易於擴充及操作。

這兩個元件會邏輯劃分正式版服務的安全性和運作需求。Confidential Space 會在高度隔離、加密及經過驗證的環境 (稱為受信任的執行環境 (TEE)) 中執行工作負載，提供必要的安全性。相較之下，MIG 提供執行大規模安全應用程式所需的必要作業功能，與 Kubernetes 類似。在單一 VM 上執行重要業務的工作負載，可能會導致速度緩慢或容易發生故障，而 MIG 可消除這類風險。這兩項措施可確保資料受到保護，且系統運作穩定。這項解決方案可確保**高可用性**和**自動修復**，因為工作負載會在集區中的多個 VM 上執行。如果其中一個 VM 當機，由於負載平衡和其餘執行個體的存在，服務仍可正常運作。

此外，MIG 會使用[可設定的健康狀態檢查](#)，持續監控 VM 的運作狀態。如果發現執行個體健康狀態不良，MIG 會自動以新的健康狀態良好 VM 取代，確保作業持續進行。

MIG 也提供[自動調度資源功能](#)，可有效調度資源。這項功能提供自動管理容量的方式，無需手動介入，可根據用量彈性新增或移除容量，滿足相關需求。

最後，MIG 可透過滾動式更新，實現零停機更新。這項功能的主要優勢是可「[一鍵升級](#)」基礎 Confidential Space OS 映像檔或應用程式的容器映像檔 (或兩者)，而且不會造成任何服務停機。MIG 會管理這項變更，逐步以執行更新後映像檔的新執行個體取代舊執行個體，確保部署期間持續可用。請注意，您的應用程式可能需要回溯相容，才能支援這類逐步升級。

3. 設定雲端資源

事前準備

1. 設定 Google Cloud 專案。如要進一步瞭解如何建立 Google Cloud 雲端專案，請參閱「[設定及瀏覽第一個 Google 專案](#)」程式碼研究室。如要瞭解如何擷取專案 ID，以及專案 ID 與專案名稱和專案編號有何不同，請參閱「[建立及管理專案](#)」。
2. 為專案[啟用帳單](#)。
3. 在其中一個 [Google 專案的 Cloud Shell](#) 中，設定必要的專案環境變數，如下所示。

```
export CURRENT_PROJECT_ID=<Google Cloud project id of current project>
```

4. 為專案啟用機密運算 API 和下列 API。

```
gcloud config set project $CURRENT_PROJECT_ID
gcloud services enable \
cloudapis.googleapis.com \
container.googleapis.com \
artifactregistry.googleapis.com \
confidentialcomputing.googleapis.com \
compute.googleapis.com \
logging.googleapis.com \
run.googleapis.com \
cloudscheduler.googleapis.com
```

5. 在 Google Cloud 專案的 Cloud Shell 中，複製 [Confidential Space Codelab Github 存放區](#)，然後使用下列指令取得完成本程式碼研究室所需的適當指令碼。

```
git clone https://github.com/GoogleCloudPlatform/confidential-space.git
```

6. 將目錄變更為執行個體群組程式碼實驗室的指令碼目錄

```
cd confidential-space/codelabs/mig_cs_codelab/scripts
```

7. 更新 config_env.sh 中的專案 ID 行，以反映所選專案的 ID。
8. 設定任何現有變數。使用這些變數覆寫資源名稱
 - 您可以使用現有的雲端資源名稱設定下列變數。如果已設定變數，系統會使用專案中對應的現有雲端資源。如未設定，雲端資源名稱會來自 config_env.sh 指令碼
9. 執行 config_env.sh 指令碼，根據資源名稱的專案 ID，將這個專案的其餘變數名稱設為值

```
source config_env.sh
```

10. 為專案新增權限。如要新增權限，請按照「[授予 IAM 角色](#)」網頁上的詳細說明操作。

您需要這項專案的下列權限

- Artifact Registry 寫入者
- Cloud Scheduler 管理員
- Compute 服務代理
- 機密運算工作負載使用者
- 記錄寫入者
- Cloud Run 開發人員
- Cloud Run 叫用者

```
gcloud config set project $CURRENT_PROJECT_ID

# Add Artifact Registry Writer role
gcloud projects add-iam-policy-binding $CURRENT_PROJECT_ID --
member="serviceAccount:${CURRENT_WORKLOAD_SERVICE_ACCOUNT}" --
role='roles/artifactregistry.writer'

# Add Confidential Space Workload User
gcloud projects add-iam-policy-binding $CURRENT_PROJECT_ID --
member="serviceAccount:${CURRENT_WORKLOAD_SERVICE_ACCOUNT}" --
role='roles/confidentialcomputing.workloadUser'

# Add Logging Log Writer
gcloud projects add-iam-policy-binding $CURRENT_PROJECT_ID --
member="serviceAccount:${CURRENT_WORKLOAD_SERVICE_ACCOUNT}" --role='roles/logging.logWriter'

# Add Cloud Run Developer
gcloud projects add-iam-policy-binding $CURRENT_PROJECT_ID --
member="serviceAccount:${CURRENT_WORKLOAD_SERVICE_ACCOUNT}" --role='roles/run.developer'

# Add Cloud Run Invoker
gcloud projects add-iam-policy-binding $CURRENT_PROJECT_ID --
member="serviceAccount:${CURRENT_WORKLOAD_SERVICE_ACCOUNT}" --role='roles/run.invoker'

# Add Cloud Scheduler Admin
gcloud projects add-iam-policy-binding $CURRENT_PROJECT_ID --
member="serviceAccount:${CURRENT_WORKLOAD_SERVICE_ACCOUNT}" --
role='roles/cloudscheduler.admin'
```

11. 查看 test_workload.py

- 查看原始碼，驗證工作負載的輸出內容，應該只會列印目前的工作負載版本
 - 我們第一次將工作負載推送至 CS 並檢查輸出內容時，應該會看到「version A」列印出來
-

步驟 4 · 約 0 分鐘

4. 設定工作負載

您必須先為本程式碼研究室中使用的工作負載建立 Docker 映像檔。這項工作負載是簡單的指令碼，會列印目前執行的工作負載版本。系統會輸出工作負載正在啟動，然後輸出工作負載版本，休眠 5 秒後，再輸出工作負載已完成。

建立工作負載的步驟

1. 執行 `create_workload.sh` 建立工作負載。這個指令碼：
 - 建立專案擁有的 Artifact Registry，工作負載會發布至該專案
 - 建構程式碼並封裝至 Docker 映像檔。詳情請參閱相關的 Dockerfile 設定[資訊](#)。
 - 將 Docker 映像檔發布至專案擁有的 Artifact Registry
 - 授予服務帳戶 <您的服務帳戶名稱> 對構件登錄 <構件登錄存放區名稱> 的讀取權限
-

5. 設定執行個體範本和代管執行個體群組

建立執行個體範本的步驟

您必須先建立執行個體範本。代管執行個體群組 (MIG) 會使用這個必要藍圖，在 Confidential Space 中佈建及執行工作負載。

執行個體範本定義了所有專用參數，因此至關重要：

- **機型**：在本範例中，我們使用支援 AMD SEV 機密運算技術 (`--confidential-compute-type=SEV`) 的 **Confidential VM** 機型 (例如 `n2d-standard-2`)。
- **VM OS 映像檔**：我們使用 `confidential-space-images` 專案和 `confidential-space-debug` 映像檔系列，提取最新的 Confidential Space 作業系統映像檔。
- **注意**：本指南使用 `debug` 映像檔，方便您進行 [疑難排解](#)。與正式版映像檔不同，偵錯版會在工作負載完成後讓 VM 繼續執行，並允許 SSH 存取以進行測試。如要使用實際的機密資料進行正式版部署，請務必切換至正式版映像檔系列。
- **工作負載參照**：中繼資料中的必要 `tee-image-reference` 行包含 Confidential Space VM 將啟動的特定容器映像檔 (您的應用程式工作負載)。

這項設定可確保代管執行個體群組：MIG 建立的每個 VM 都是經過適當設定的 Confidential Space，隨時可執行工作負載。

建立代管執行個體群組的步驟

下一步是使用您剛定義的範本，建立代管執行個體群組 (MIG)。MIG 可自動部署、管理及調度多個相同的 VM，因此非常重要。

`create_launch_mig.sh` 指令碼可達成三個主要目標：

1. 建立 MIG

- **指令**：`gcloud compute instance-groups managed create ${CURRENT_MIG_NAME}`
- **用途**：這個指令會建立群組，用於管理 VM。
- `--size 3`：指定 MIG 應先建立並維護 **3** 個工作負載執行個體。
- `--template ${TEMPLATE_NAME}`：請務必參照先前建立的執行個體範本，確保所有 3 個執行個體都設定為 Confidential Space VM，並執行特定 `tee-image-reference` 工作負載容器。
- `--zone ${CURRENT_PROJECT_ZONE}`：指定執行個體的部署位置。

2. 擷取專案編號

- **指令**：`PROJECT_NUMBER=$(gcloud projects describe ${PROJECT_ID} --format="value(projectNumber)")`
- **用途**：這個指令碼會擷取專案的數值 ID。建立服務帳戶角色和權限時，通常需要這個號碼，尤其是 Google 管理的服務代理。

3. 授予 IAM 權限

- **指令**：`gcloud projects add-iam-policy-binding --role="roles/compute.serviceAgent"`
- **用途**：這個步驟會將 **Compute Engine 服務代理人** 角色授予工作負載的服務帳戶 (`${SERVICE_ACCOUNT}`)。這項權限非常重要，因為服務帳戶可藉此代表專案的 Compute Engine 服務執行動作，這通常是 MIG 自動化功能 (例如管理執行個體、設定網路，以及與其他 Google Cloud 服務互動) 的必要條件。

執行 `create_launch_mig.sh` 建立代管執行個體群組。

6. 啟用自動修復和自動調度資源的步驟

設定自動修復功能

為確保高可用性，我們會驗證工作負載是否能正常回應。如果應用程式凍結，MIG 應替換 VM。如要瞭解來源 IP 範圍的防火牆規則，請參閱[這份文件](#)。

```
# 1. Create Health Check (TCP Port 22)
gcloud compute health-checks create tcp ${HEALTH_CHECK_NAME} \
  --port 22 \
  --check-interval 30s \
  --healthy-threshold 1 \
  --timeout 10s \
  --unhealthy-threshold 3 \
  --global

# 2. Allow Health Check Traffic (Firewall)
gcloud compute firewall-rules create allow-health-check \
  --allow tcp:22 \
  --source-ranges 130.211.0.0/22,35.191.0.0/16 \
  --network default \
  --project="${CURRENT_PROJECT_ID}" \

# 3. Apply to MIG
gcloud compute instance-groups managed update ${CURRENT_MIG_NAME} \
  --health-check ${HEALTH_CHECK_NAME} \
  --initial-delay 60 \
  --zone ${CURRENT_PROJECT_ZONE}
```

設定自動調度資源功能

我們會將群組設定為自動在 1 到 5 個執行個體之間調度資源，以因應流量高峰。

```
gcloud compute instance-groups managed set-autoscaling ${CURRENT_MIG_NAME} \
  --max-num-replicas 5 \
  --target-cpu-utilization 0.80 \
  --cool-down-period 90 \
  --zone ${CURRENT_PROJECT_ZONE}
```

7. 驗證工作負載及設定映像檔更新

驗證工作負載

代管執行個體群組 (MIG) 啟動 VM 後，我們需要確認 Confidential Space 工作負載是否正常運作。

您可以使用 Google Cloud 控制台或指令列完成這項操作。

```
gcloud compute instance-groups managed list-instances ${CURRENT_MIG_NAME} \
  --zone ${CURRENT_PROJECT_ZONE}
```

您也可以檢查該特定執行個體的序列埠輸出內容，查看工作負載的記錄

```
# Replace <INSTANCE_NAME> with one of the names from the previous command
gcloud compute instances get-serial-port-output <INSTANCE_NAME> \
  --zone ${CURRENT_PROJECT_ZONE} \
  --port 1
```

設定圖片更新

在正式環境中，您必須定期更新代管執行個體群組 (MIG)，以因應下列兩種不同情境：

1. **工作負載更新**：發布新版應用程式程式碼 (例如將 test_workload.py 從 v1 更新至 v2)。
2. **基礎架構更新**：Google 發布 Confidential Space 基礎 OS 的安全性修補程式或更新。請注意，最佳做法是至少每月擷取最新的 CS 映像檔。

由於我們使用動態映像檔連結 (.../images/family/...) 和動態容器標記 (:latest) 設定執行個體範本，因此只要執行一次「滾動式取代」作業，即可處理這兩種情況。這可確保 VM 機群一律執行最新堆疊，不會停機，也不必為每次小幅變更建立新的執行個體範本。

輪替取代指令碼

在 `update_images` 目錄下，前往 `update_images_script.sh`。這個指令碼會觸發滾動式取代作業，逐步毀損並重新建立群組中的每個 VM

```
#!/bin/bash

# Initialize the template
gcloud compute instance-groups managed set-instance-template "${CURRENT_MIG_NAME}" \
--template=projects/"${PROJECT_ID}"/global/instanceTemplates/"${TEMPLATE_NAME}" \
--zone="${CURRENT_PROJECT_ZONE}" \
--project="${PROJECT_ID}"

# Trigger the rolling replace
gcloud compute instance-groups managed rolling-action replace "${CURRENT_MIG_NAME}" \
--version=template="${TEMPLATE_NAME}" \
--project="${PROJECT_ID}" \
--zone="${CURRENT_PROJECT_ZONE}" \
--max-surge=1 \
--max-unavailable=0

# Wait for the update to complete
gcloud compute instance-groups managed wait-until --version-target-reached
"${CURRENT_MIG_NAME}" \
--zone="${CURRENT_PROJECT_ZONE}" \
--project="${PROJECT_ID}"
```

這個指令碼可以使用取代，而非重新啟動。

- 「重新啟動」只會重新啟動裝置，這會保留現有的 OS 磁碟，也就是**不會**取得新的 OS 修補程式。
- 「取代」會刪除 VM，並根據範本建立新的 VM。這會強制系統從系列中查詢最新的 Confidential Space OS 映像檔，並從登錄檔中提取「最新」容器映像檔。

--max-surge=1：這可讓 MIG 暫時目標大小之上建立 **1 個額外 VM**。系統會啟動新的 (更新後) VM，並等待該 VM 正常運作後，再刪除舊的 (過時) VM。

--max-unavailable=0：確保完全不需要停機。這會告知 MIG，除非已成功完成替代機器的激增，否則**不得**將任何機器離線。

滾動式重新啟動指令碼

目錄 `update_images` 下還有另一個指令碼 `update_workload_image_script.sh`。這個指令碼會觸發**輪流重新啟動**，這是專門用於重新整理工作負載的快速方法。由於 Confidential Space 會在每次啟動時從登錄檔提取容器映像檔，因此只要重新啟動，應用程式就會更新為 `:latest` 版本，不必變更基礎主機。

```
#!/bin/bash
# Reboots the existing VMs to refresh the container
gcloud compute instance-groups managed rolling-action restart "${CURRENT_MIG_NAME}" \
  --project="${PROJECT_ID}" \
  --zone="${CURRENT_PROJECT_ZONE}" \
  --max-surge=1 \
  --max-unavailable=0

# Wait for the update to complete
gcloud compute instance-groups managed wait-until --stable "${CURRENT_MIG_NAME}" \
  --zone="${CURRENT_PROJECT_ZONE}" \
  --project="${CURRENT_PROJECT_ID}"
```

驗證更新後的工作負載

我們可以模擬實際發布應用程式，測試「一鍵升級」功能。我們會修改工作負載程式碼、推送至 Artifact Registry、更新 MIG，並確認新版本在零停機的情況下執行。

步驟 1：部署新的工作負載版本

首先，我們需要建立應用程式的「新」版本。

1. 開啟本機的 test_workload.py 檔案。
2. 將版本列印陳述式從 print("Workload Version A") 變更為 print("Workload Version B")
3. 執行 create_workload.sh，重新建構容器映像檔並推送至 Artifact Registry。請注意，我們正在推送至相同標記 (:latest)。

步驟 2：執行滾動更新

執行我們在前一節建立的更新指令碼。這會強制 MIG 替換每個 VM，並提取與 :latest 相關聯的新容器雜湊。

```
# Run your update script
./update_images/update_images_script.sh
```

等待指令碼完成

步驟 3：透過序列埠驗證更新

更新完成後，我們會確認新的 VM 是否正在執行更新後的程式碼。

```
# Replace <INSTANCE_NAME> with one of the names from the previous command
gcloud compute instances get-serial-port-output <INSTANCE_NAME> \
  --zone ${CURRENT_PROJECT_ZONE} \
  --port 1
```

取得新執行個體的名稱：

```
gcloud compute instance-groups managed list-instances ${CURRENT_MIG_NAME} --zone
${CURRENT_PROJECT_ZONE}
```

檢查記錄：

```
# Replace <NEW_INSTANCE_NAME> with one of the names of the running VMs
gcloud compute instances get-serial-port-output <NEW_INSTANCE_NAME> \
  --zone ${CURRENT_PROJECT_ZONE} \
  --port 1
```

執行個體啟動後，請從先前的 gcloud 指令中選取任一執行個體名稱，即可查看其序列埠

預期輸出內容：您應該會看到更新後的記錄訊息，確認部署作業已順利完成：

... Workload Version B ...

步驟 4：驗證基礎架構設定 (選用)

您也可以檢查執行個體範本的中繼資料，確認範本已正確設定，可提取作業系統和工作負載的動態更新。

執行下列指令，查看動態容器參照：

```
gcloud compute instance-templates describe ${TEMPLATE_NAME} \
  | grep -A 1 tee-image-reference
```

結果：您應該會看到容器映像檔結尾為「:latest」。

- **影響：**由於範本指向代碼，而非特定雜湊，因此每次滾動式取代作業都會成功提取您在步驟 1 中推送的最新程式碼。

(選用) 自動更新

手動更新適用於主要版本發布，但您通常會希望機群自動取得最新的安全性修補程式或一般部署版本，不必人為介入。

我們可以將更新指令碼封裝到 Cloud Run 工作中，自動執行「滾動式取代」程序。在本程式碼研究室中，我們會每 15 分鐘觸發一次。在正式環境中，執行頻率應大幅降低。使用者可依需求設定每週或每月執行一次。

步驟 1：將更新程式指令碼容器化

首先，我們需要將 update_images_script.sh (其中包含 gcloud ... rolling-action replace 邏輯) 封裝到 Docker 容器中，以便在雲端執行。

我們準備了輔助指令碼，可建構這個容器並推送至 Artifact Registry。

執行下列指令：

```
# Build and Push the "Updater" Container
# This packages your update logic into a docker image
./update_images/deploy_docker_script_image.sh
```

用途：

- 這個指令會從 update_images/ 目錄取得 update_images_script.sh。
- 這會建立包含 Google Cloud SDK 和指令碼的 Docker 映像檔。
- 這個指令會將映像檔推送至 \${CURRENT_PROJECT_REGION}-docker.pkg.dev/\${PROJECT_ID}/\${REPOSITORY}/update-script:latest。

步驟 2：部署及排定工作時間

現在我們需要告知 Google Cloud 定期執行這個容器。我們會使用 **Cloud Run Jobs** 執行容器，並使用 **Cloud Scheduler** 觸發容器。

執行排程設定指令碼：

```
# Create the Cloud Run Job and the Scheduler Trigger
./create_configs/create_schedule_job.sh
```

指令碼內容：這段指令碼會執行兩項重要動作：

1. 建立 **Cloud Run** 工作：定義名為 `mig-updater-job` 的工作，執行我們剛推送的容器。
2. 建立 **Cloud Scheduler** 觸發條件：設定 Cloud Scheduler 工作，每 15 分鐘呼叫一次 Cloud Run Job API。

```
# (Snippet from create_schedule_job.sh for reference)
# The schedule is set to run every 15 minutes for testing purposes
gcloud scheduler jobs create http ${SCHEDULER_NAME} \
  --schedule "*/15 * * * *" \
  --uri "https://${CURRENT_PROJECT_REGION}-
run.googleapis.com/apis/run.googleapis.com/v1/namespaces/${PROJECT_ID}/jobs/${JOB_NAME}:run"
\
  --http-method POST \
  --oauth-service-account-email ${SERVICE_ACCOUNT}
```

步驟 3：驗證自動化

不必等待 15 分鐘即可測試。您可以強制排程器立即執行，以驗證管道。

1. 強制執行工作：

```
gcloud scheduler jobs run ${SCHEDULER_NAME} --location ${CURRENT_PROJECT_REGION}
```

2. 檢查執行作業：前往 **Cloud Run** 控制台 > 「Jobs」(作業)。您應該會看到新的執行作業開始。
3. 檢查 **MIG**：執行 `gcloud compute instance-groups managed list-instances ${CURRENT_MIG_NAME}`。作業觸發滾動式更新時，您會看到執行個體進入「RECREATING」狀態。

為什麼是 15 分鐘？在本程式碼研究室中，我們將排程設為 `*/15 * * * *`，方便您快速查看結果。在實際正式環境中，您可能會將這項設定變更為每日執行 (例如 `0 3 * * *` 代表凌晨 3 點)，或每週執行。

8. 清除

您可以使用 `cleanup.sh` 清理指令碼，清除在本程式碼研究室中建立的資源。在這次清理作業中，系統會刪除下列資源：

- 代管執行個體群組 (`${CURRENT_MIG_NAME}`) 及其基礎 VM。
- 執行個體範本 (`${TEMPLATE_NAME}`)。
- 健康狀態檢查和防火牆規則 (`${HEALTH_CHECK_NAME}`)。
- Artifact Registry 存放區 (`${REPOSITORY}`)。
- 服務帳戶 (如果您為本實驗室建立專用帳戶)。

探索完畢後，請按照「[關閉 \(刪除\) 專案](#)」一文的說明刪除專案。

恭喜

恭喜，您已成功完成本程式碼研究室！

您已瞭解如何使用代管執行個體群組 (MIG)，安全地調度機密空間工作負載的資源。您已成功設定**自動修復**功能，可從故障中復原；設定**自動調度資源**功能，可處理流量尖峰；並對您的 Confidential Space OS 映像檔和工作負載容器執行**零停機時間更新**。

後續步驟

查看其他 Confidential Space 程式碼研究室：

- [使用 Confidential Space 保護機器學習模型和智慧財產](#)
- [如何透過多方運算和 Confidential Space 進行數位資產交易](#)
- [使用 Confidential Space 分析機密資料](#)
- [搭配使用 Confidential Space 與未儲存於雲端服務供應商的受保護資源](#)

其他資訊

- [根據 Monitoring 指標調度資源](#)
 - [感到孤單嗎？機密運算來救援](#)
 - [GCP 的機密運算](#)
 - [Confidential Space：保護隱私權的協作方式，引領未來趨勢](#)
 - [Confidential Space 安全總覽](#)
 - [強化資料隱私權：大規模進行多方機密協作](#)
-